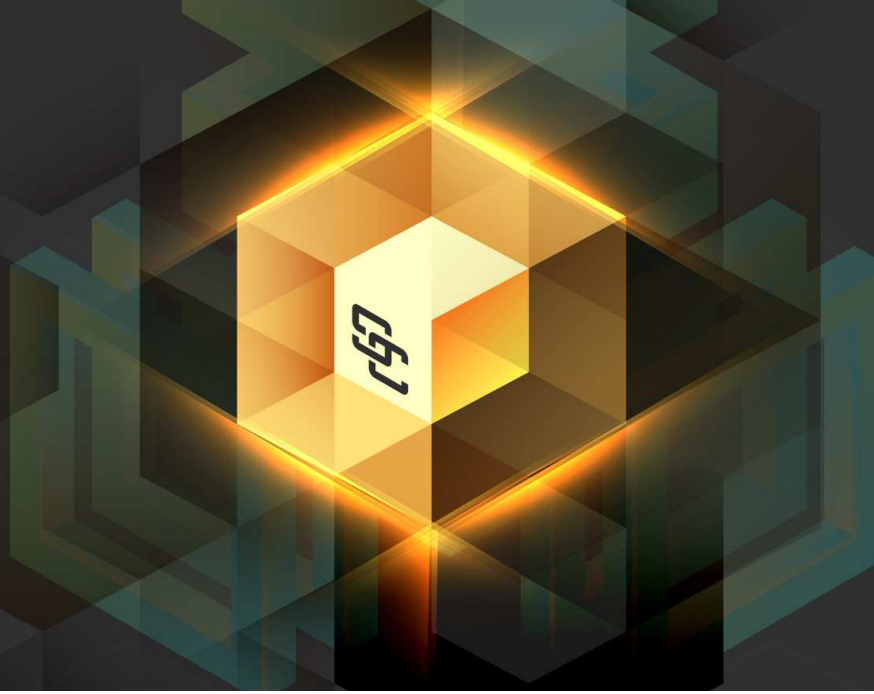




Security Audit

Full Audit Report For BIGRIG



Client: BIGRIG

<https://www.bigrigtoken.com/>

January 25, 2026

Version: 1.0

Table of Contents

- 1 Disclaimer 3
- 2 Security Assessment Procedure 4
- 3 Risk Rating 5
- 4 Category 6
- 5 Executive Summary 7
- 6 Audit Information 8
- 7 Findings List 12
- 8 Findings 13
- A Appendix 27
 - A.1 Source Units in Scope 27
 - A.2 Visibility, Mutability, Modifier function testing 28
 - A.3 Contracts Description Table 29
 - A.4 Call Graph 35
 - A.5 UML Class Diagram 36



1 Disclaimer

This security assessment does not guarantee the security of the program instruction received from the client, herein referred to as "Source code."

The Service Provider will not be held liable for any legal liability arising from errors in the security assessment. The responsibility for ensuring the security of the program instruction will be solely with the Client, herein referred to as "Service User." The Service User agrees not to hold the Service Provider liable for any such liability.

By contract, the Service Provider is committed to conducting security assessments with integrity, professional ethics, and transparency in delivering security assessments to users. The Service Provider reserves the right to postpone the delivery of the security assessment if necessary, regardless of the reason for the delay. The Service Provider will not be held responsible for any delayed security assessments.

If the Service Provider identifies a vulnerability, we will notify the Service User via a Preliminary Report, which will be maintained in confidence for security purposes. The Service Provider disclaims responsibility in the event of any attacks occurring before or after conducting a security assessment. All responsibility for ensuring the security of the program instruction will be solely with the Service User.

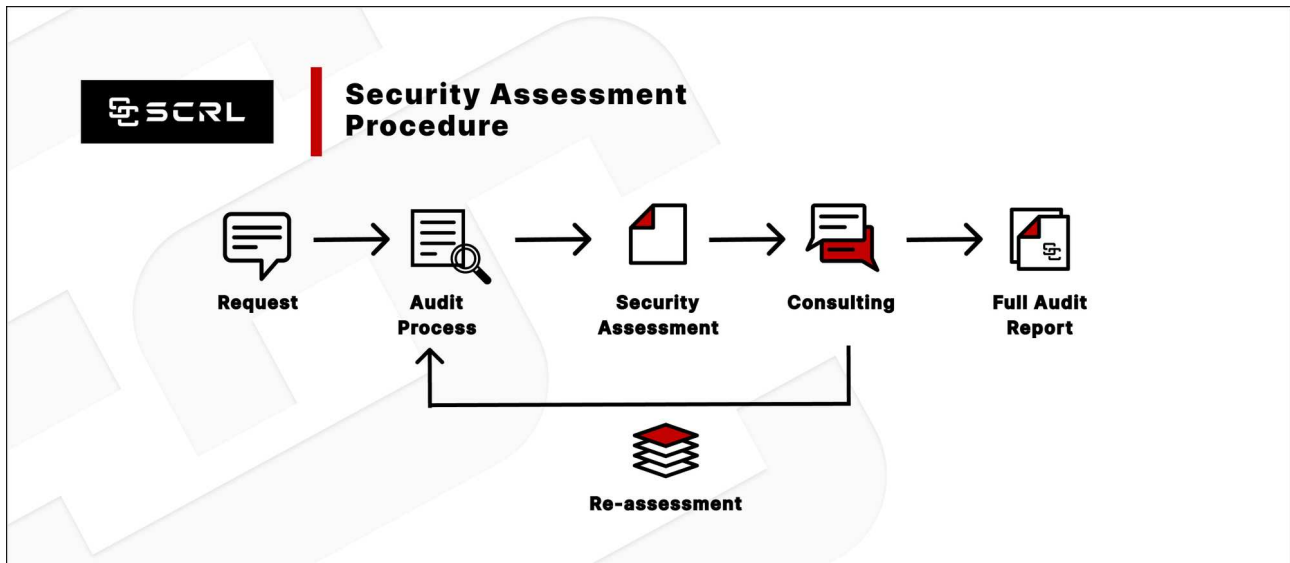
Please be advised that the Security Assessment is not an investment advisory document and should not be interpreted as such. We hereby disclaim any liability arising from any investment:

- Security Assessment Is **Not Financial/Investment Advice** Any loss arising from any investment in any project is the responsibility of the investor.
- SCRL **disclaims** any liability incurred, whether it's **Rugpull, Abandonment, Soft Rugpull, Exploit, Exit Scam.**
- **Cryptocurrency and digital token involve high risks; investors may lose all investment money and should study information carefully and make investments according to own risk profile.**
- Please thoroughly comprehend any aspect of a project, including liquidity checks and pre-sale stages, and assume the associated risks independently. We hereby disclaim any liability for any investment decisions made by you.

For full Legal / TOS / Privacy Policy please visit: <https://scrl.io/legal>



2 Security Assessment Procedure



- **Request:** The client must submit a formal request and follow the procedure. By submitting the source code and agreeing to the terms of service.
- **Audit Process:** Check for vulnerabilities and vulnerabilities from source code obtained by experts using formal verification methods, including using powerful tools such as Static Analysis, SWC Registry, Dynamic Security Analysis, Automated Security Tools, CWE, Syntax & Parameter Check with AI ,WAS (Warning Avoidance System a python script tools powered by SCRL).
- **Security Assessment:** Deliver Preliminary Security Assessment to clients to acknowledge the risks and vulnerabilities.
- **Consulting:** Discuss on risks and vulnerabilities encountered by clients to apply to their source code to mitigate risks.
- **Re-assessment:** Reassess the security when the client implements the source code improvements and if the client is satisfied with the results of the audit. We will proceed to the next step.
- **Full Audit Report:** SCRL provides clients with official security assessment reports informing them of risks and vulnerabilities. Officially and it is assumed that the client has been informed of all the information.



3 Risk Rating

Risk rating using this commonly defined: **Risk rating = impact * confidence**

- **Impact:** The severity and potential impact of an attacker attack
- **Confidence:** Ensuring that attackers expose and use this vulnerability

Confidence & Impact [Likelihood]	Low	Medium	High
Low	Informational	Low	Medium
Medium	Low	Medium	High
High	Medium	High	Critical

Severity is a risk assessment It is calculated from the Impact and Confidence values using the following calculation methods, Risk rating = impact * confidence It is categorized into 7 categories severity based

Severity Risk	Description
Critical	Critical severity is assigned to security vulnerabilities that pose a severe threat to the smart contract and the entire blockchain ecosystem.
High	High-severity issues should be addressed quickly to reduce the risk of exploitation and protect users' funds and data.
Medium	It's essential to fix medium-severity issues in a reasonable timeframe to enhance the overall security of the smart contract.
Low	While low-severity issues can be less urgent, it's still advisable to address them to improve the overall security posture of the smart contract.
Informational	Used to categorize security findings that do not pose a direct security threat to the smart contract or its users. Instead, these findings provide additional information, recommendations



4 Category

Category	Description
Centralization	Centralization Risk is The risk incurred by a sole proprietor, such as the Owner being able to change something without permission
Economics Risk	Economics Risk is Risks that may affect the economic mechanism system, such as the ability to increase Mint token
Logical Issue	Logical Issue is that can cause errors to core processing, such as any prior operations that cause background processes to crash.
Authorization	Authorization is Possible pitfalls from weak coding allows unrelated people to take any action to modify the values.
Mathematical	Any erroneous arithmetic operations affect the operation of the system or lead to erroneous values.
Naming Conventions	naming variables that may affect code understanding or naming inconsistencies
Security Risk	Security Risk of loss or damage if it's no mitigate
Coding Style	Coding Style is Tips coding for efficiency performance
Best Practices	Best Practices is suggestions for improvement
Optimization	Optimization is performance improvement
Gas Optimization	Gas Optimization is increase performance to avoid expensive gas
Dead Code	Dead Code having unused code This may result in wasted resources and gas fees.
Input Validation	Proper input validation is essential to ensure that a smart contract processes only valid and anticipated data. Failure to implement robust input validation mechanisms may lead to various security vulnerabilities, including logic manipulation, unauthorized access, and unintended contract behavior.



5 Executive Summary

For this security assessment, SCRL received a request on **January 22, 2026**



Security Assessment by SCRL

Full Audit Report For BIGRIG

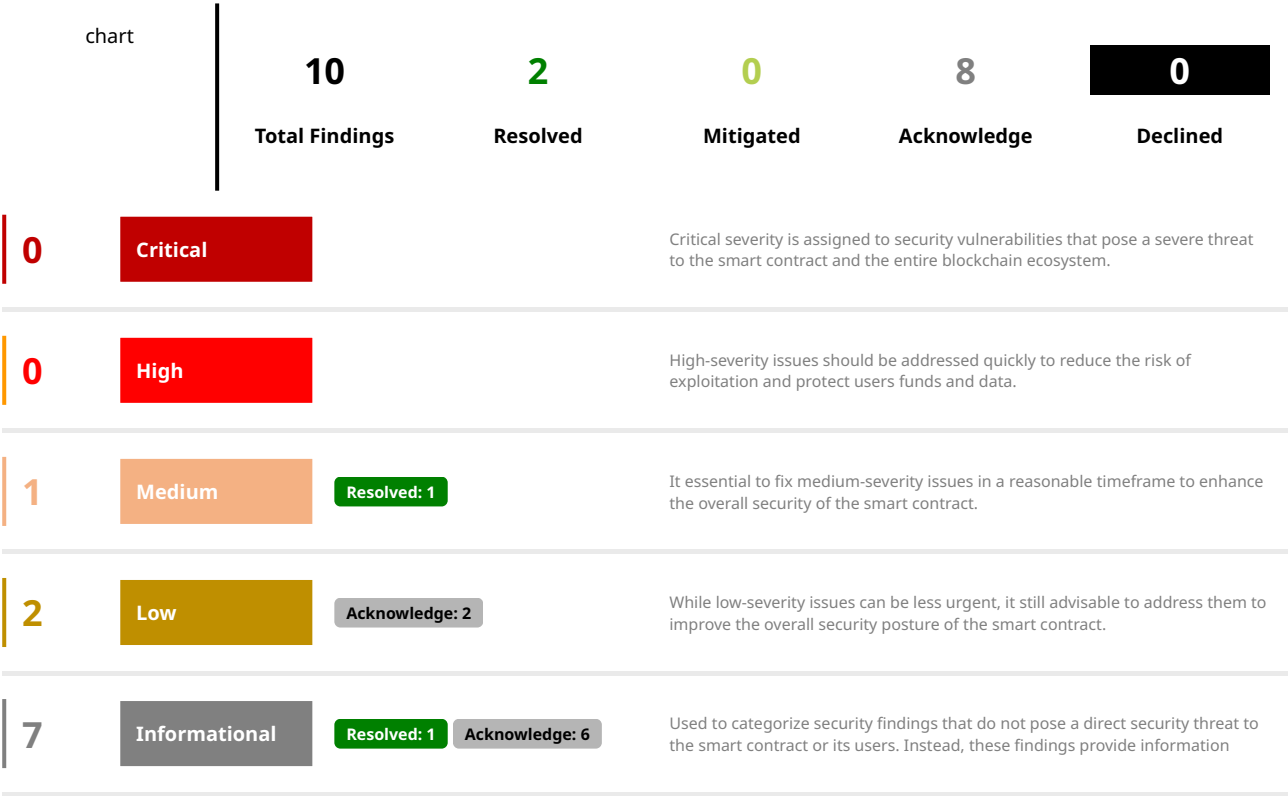
January 25, 2026
<https://scrl.io/project/bigrig>



Audit Score	Project Website	X (Twitter)	Telegram
9.2	https://www.bigrigtoken.com/	https://x.com/BIGRIG_TOKEN	https://t.me/brig_united_tip30

Client	Confidential	Audit Method	Language
BIGRIG	Public	Static Analysis + Dynamic Analysis + Manual Review + Formal Verification + Hyper Fuzzing + WAS (Custom Detector)	Solidity

Vulnerabilities Summary:



6 Audit Information

Audit Scope:

File	SHA1-Hash
src/ LiquidityGeneratorToken.sol	312c1144b87ed7bc1ecb63b5492d42cf21beffb6

Onchain Scope:

Chain	Contract Address
BNB Chain	0x15bEeb48823f790B4F023525f979030b70AD9450

Security Assessment Author:

Name	Role
Chinnakit J.	CEO & Founder
Ronny C.	CTO & Head of Security Researcher
Mark K.	Security Researcher

Smart Contract Audit Summary:



The results of the security assessment revealed

No Critical Vulnerabilities.

Full Audit Report For BIGRIG
January 25, 2026



Digital Sign:



Background pattern of repeating 'SCRL' text.

ID: 61099204-B342-42D8-8506-7576B3E0CCE0
Digitally signed by <contact@scrl.io>
January 25, 2026 01:06 PM +07



SWC Checklist

ID	Title	Scanning	Result
SWC-100	Function Default Visibility	Completed	No Risk
SWC-101	Integer Overflow and Underflow	Completed	No Risk
SWC-102	Outdated Compiler Version	Completed	No Risk
SWC-103	Floating Pragma	Completed	No Risk
SWC-104	Unchecked Call Return Value	Completed	No Risk
SWC-105	Unprotected Ether Withdrawal	Completed	No Risk
SWC-106	Unprotected SELFDESTRUCT Instruction	Completed	No Risk
SWC-107	Reentrancy	Completed	No Risk
SWC-108	State Variable Default Visibility	Completed	No Risk
SWC-109	Uninitialized Storage Pointer	Completed	No Risk
SWC-110	Assert Violation	Completed	No Risk
SWC-111	Use of Deprecated Solidity Functions	Completed	No Risk
SWC-112	Delegatecall to Untrusted Callee	Completed	No Risk
SWC-113	DoS with Failed Call	Completed	No Risk
SWC-114	Transaction Order Dependence	Completed	No Risk
SWC-115	Authorization through tx.origin	Completed	No Risk
SWC-116	Block values as a proxy for time	Completed	No Risk
SWC-117	Signature Malleability	Completed	No Risk
SWC-118	Incorrect Constructor Name	Completed	No Risk
SWC-119	Shadowing State Variables	Completed	No Risk
SWC-120	Weak Sources of Randomness from Chain Attributes	Completed	No Risk
SWC-121	Missing Protection against Signature Replay Attacks	Completed	No Risk



ID	Title	Scanning	Result
SWC-122	Lack of Proper Signature Verification	Completed	No Risk
SWC-123	Requirement Violation	Completed	No Risk
SWC-124	Write to Arbitrary Storage Location	Completed	No Risk
SWC-125	Incorrect Inheritance Order	Completed	No Risk
SWC-126	Insufficient Gas Griefing	Completed	No Risk
SWC-127	Arbitrary Jump with Function Type Variable	Completed	No Risk
SWC-128	DoS With Block Gas Limit	Completed	No Risk
SWC-129	Typographical Error	Completed	No Risk
SWC-130	Right-To-Left-Override control character (U+202E)	Completed	No Risk
SWC-131	Presence of unused variables	Completed	No Risk
SWC-132	Unexpected Ether balance	Completed	No Risk
SWC-133	Hash Collisions With Multiple Variable Length Arguments	Completed	No Risk
SWC-134	Message call with hardcoded gas amount	Completed	No Risk
SWC-135	Code With No Effects	Completed	No Risk
SWC-136	Unencrypted Private Data On-Chain	Completed	No Risk



OWASP Smart Contract Top 10 2025 Checklist

ID	Title	Scanning	Result
SC01:2025	Access Control Vulnerabilities	Completed	No Risk
SC02:2025	Price Oracle Manipulation	Completed	No Risk
SC03:2025	Logic Errors	Completed	No Risk
SC04:2025	Lack of Input Validation	Completed	Risky
SC05:2025	Reentrancy Attacks	Completed	No Risk
SC06:2025	Unchecked External Calls	Completed	No Risk
SC07:2025	Flash Loan Attacks	Completed	No Risk
SC08:2025	Integer Overflow and Underflow	Completed	No Risk
SC09:2025	Insecure Randomness	Completed	No Risk
SC10:2025	Denial of Service (DoS) Attacks	Completed	No Risk



7 Findings List

chart



This security assessment report for **BIGRIG** a total of **10 vulnerabilities**. The evaluation was conducted using the **Static Analysis + Dynamic Analysis + Manual Review + Formal Verification + Hyper Fuzzing + WAS (Custom Detector)** security assessment methodology.

ID	Vulnerabilities	Severity	Category	Status
CEN-01	Tax Fee Can Be Set to 100%	Medium	Centralization	Resolved
MIV-01	Missing Zero Address Validation	Low	Input Validation	Acknowledge
MEA-01	Missing Events for Arithmetic State Changes (events-maths)	Low	Best Practices	Acknowledge
CEN-02	Centralization Risk	Informational	Centralization	Resolved
COL-01	Costly operations in a loop (costly-loop)	Informational	Best Practices	Acknowledge
SNC-01	Conformity to Solidity naming conventions	Informational	Coding Style	Acknowledge
UCE-01	Use Custom Errors	Informational	Gas Optimization	Acknowledge
LRS-01	Use of Long Revert Strings in `require()` or `revert()`	Informational	Gas Optimization	Acknowledge
USR-01	Use shift Right/Left instead of division/multiplication if possible	Informational	Gas Optimization	Acknowledge
UIC-01	Use != 0 instead of > 0 for unsigned integer comparison	Informational	Gas Optimization	Acknowledge



8 Findings

CEN-01: Tax Fee Can Be Set to 100%

Vulnerabilities	Severity	Category	Status
Tax Fee Can Be Set to 100%	Medium	Centralization	Resolved

Finding Code

```
uint256 tFee = calculateTaxFee(tAmount); // tAmount * taxFeeBps / 10^4
uint256 tTransferAmount = tAmount.sub(tFee); // Reverts if tFee > tAmount
```

Description

The contract allows the owner to arbitrarily set the `_taxFee` parameter via the `setTaxFeePercent(uint256 taxFeeBps)` function without enforcing a maximum individual cap. Although the total fee (`_taxFee + _liquidityFee + _charityFee`) is restricted to 25% (2500 bps), there is no restriction on the individual `_taxFee` value.

This enables the owner to call `setTaxFeePercent(10000)` (100%) or even higher values (e.g., 50000 = 500%), causing all token transfers to revert due to arithmetic underflow in the fee calculation logic.

When `_taxFee >= 10000`, `tFee >= tAmount`, resulting in `tTransferAmount = 0` or a revert due to underflow (in Solidity 0.8+). This renders the token completely untradeable for all non-excluded addresses.

Recommendation

In terms of timeframes, there are three categories: short-term, long-term, and permanent.

For short-term solutions, a combination of timelock and multi-signature (2/3 or 3/5) can be used to mitigate risk by delaying sensitive operations and avoiding a single point of failure in key management. This includes implementing a timelock with a reasonable latency, such as 48 hours, for privileged operations; assigning privileged roles to multi-signature wallets to prevent private key compromise; and sharing the timelock contract and multi-signer addresses with the public via a medium/blog link.

For long-term solutions, a combination of timelock and DAO can be used to apply decentralization and transparency to the system. This includes implementing a timelock with a reasonable latency, such as 48 hours, for privileged operations; introducing a DAO/governance/voting module to increase transparency and user involvement; and sharing the timelock contract, multi-signer addresses, and DAO information with the public via a medium/blog link.

Finally, permanent solutions should be implemented to ensure the ongoing security and protection of the system.

- Update `setTaxFeePercent()` to enforce a hard cap on individual tax fee:

```
uint256 public constant MAX_TAX_FEE = 2500; // 25% max individual tax

function setTaxFeePercent(uint256 taxFeeBps) external onlyOwner {
    require(taxFeeBps <= MAX_TAX_FEE, "Tax fee cannot exceed 25%");
    require(
        taxFeeBps + _liquidityFee + _charityFee <= MAX_FEE,
        "Total fee exceeds 25%"
    );
};
```



```
_previousTaxFee = _taxFee;  
_taxFee = taxFeeBps;  
}
```

OR

- Call `renounceOwnership()` immediately after deployment to remove central control.

Alleviation

BIGRIG Team has already resolved this issue by set MAX_FEE is 2500 (25%) and then renounced ownership to `0x0000000000000000000000000000000000000000000000000000000000000000` at transaction <https://bscscan.com/tx/0x250b3e15b1a5127f75f095a554ffe364c920b01e72d34ace40faae6f27499180>

References

--



MIV-01: Missing Zero Address Validation

Vulnerabilities	Severity	Category	Status
Missing Zero Address Validation	Low	Input Validation	Acknowledge

Finding Code



LiquidityGeneratorToken.constructor(string,string,uint256,address,address,uint16,uint16,uint16,address,uint256).serviceFeeReceiver_ (src/LiquidityGeneratorToken.sol:946-948) lacks a zero-check on :

- address(serviceFeeReceiver_).transfer(serviceFee_) (src/LiquidityGeneratorToken.sol#1008-1010)

Description

The contract does not implement a validation check to prevent the usage of the zero address (0x00) in functions

Impact: Proper input validation is essential to ensure that a smart contract processes only valid and anticipated data. Failure to implement robust input validation mechanisms may lead to various security vulnerabilities, including logic manipulation, unauthorized access, and unintended contract behavior.

Recommendation

Ensure all inputs are strictly validated for type, range, structure, and caller authorization, with clear error handling to prevent unauthorized access and unintended behavior.

We recommend that if you are using 0 input address, you should check for example:

```
require(!!! != address(0), "Invalid address");
```

Note: Please change xxx to your function parameter.

References

SC04:2025 - Lack of Input Validation: <https://owasp.org/www-project-smart-contract-top-10/2025/en/src/SC04-lack-of-input-validation.html>

SWC-135: Code With No Effects: <https://swcregistry.io/docs/SWC-135/>



MEA-01: Missing Events for Arithmetic State Changes (events-maths)

Vulnerabilities	Severity	Category	Status
Missing Events for Arithmetic State Changes (events-maths)	Low	Best Practices	Acknowledge

Finding Code

✖

LiquidityGeneratorToken.setCharityFeePercent(uint256) (src/LiquidityGeneratorToken.sol:1217-1224) should emit an event for:

- _charityFee = charityFeeBps (src/LiquidityGeneratorToken.sol#1220-1221)

✖

LiquidityGeneratorToken.setLiquidityFeePercent(uint256) (src/LiquidityGeneratorToken.sol:1204-1217) should emit an event for:

- _liquidityFee = liquidityFeeBps (src/LiquidityGeneratorToken.sol#1210-1211)

✖

LiquidityGeneratorToken.setTaxFeePercent(uint256) (src/LiquidityGeneratorToken.sol:1199-1204) should emit an event for:

- _taxFee = taxFeeBps (src/LiquidityGeneratorToken.sol#1200)

Description

The contract performs **arithmetic operations that modify critical state variables** without emitting corresponding **events**.

In Solidity, emitting events when state-changing arithmetic occurs is considered a **best practice** because it improves **on-chain transparency**, allows for easier **off-chain monitoring**, and facilitates debugging or analytics.

Without events, users and external systems (e.g., block explorers, dApps, monitoring tools) may find it difficult to track important state changes, even though they occur correctly on-chain.

Impact:

- Reduced transparency of state changes
- Difficulty for external systems to index and monitor critical operations
- Potential compliance and auditability issues

Recommendation

Emit appropriate **events** whenever critical arithmetic state updates occur. This aligns with ERC standards and industry best practices.

References

Solidity Events Documentation <https://docs.soliditylang.org/en/latest/contracts.html#events>



CEN-02: Centralization Risk

Vulnerabilities	Severity	Category	Status
Centralization Risk	Informational	Centralization	Resolved

Finding Code

```
File: LiquidityGeneratorToken.sol

165: function renounceOwnership() public virtual onlyOwner {
173: function transferOwnership(address newOwner) public virtual onlyOwner {
1186: function excludeFromReward(address account) public onlyOwner {
1196: function includeInReward(address account) external onlyOwner {
1233: function excludeFromFee(address account) public onlyOwner {
1237: function setTaxFeePercent(uint256 taxFeeBps) external onlyOwner {
1245: function setLiquidityFeePercent(uint256 liquidityFeeBps) external onlyOwner {
1256: function setCharityFeePercent(uint256 charityFeeBps) external onlyOwner {
1264: function setSwapBackSettings(uint256 _amount) external onlyOwner {
```

Description

Centralization Risk



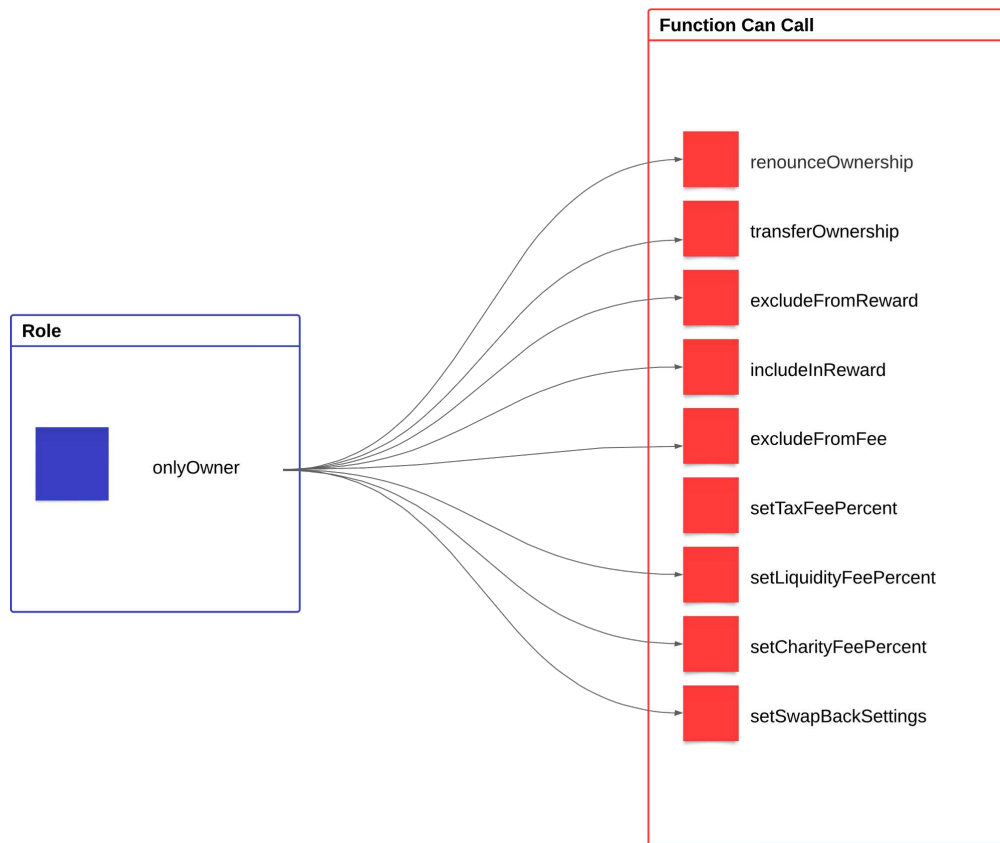


Figure 1 - This image shows who has privileges to call the function.

A Centralization Risk has been identified for the `Contract` entity, which has privileges set to `onlyOwner`. This restriction allows the execution of the following functions: `function`

Each function possesses the following capabilities:

Recommendation

In terms of timeframes, there are three categories: short-term, long-term, and permanent.

For short-term solutions, a combination of timelock and multi-signature (2/3 or 3/5) can be used to mitigate risk by delaying sensitive operations and avoiding a single point of failure in key management. This includes implementing a timelock with a reasonable latency, such as 48 hours, for privileged operations; assigning privileged roles to multi-signature wallets to prevent private key compromise; and sharing the timelock contract and multi-signer addresses with the public via a medium/blog link.

For long-term solutions, a combination of timelock and DAO can be used to apply decentralization and transparency to the system. This includes implementing a timelock with a reasonable latency, such as 48 hours, for privileged operations; introducing a DAO/governance/voting module to increase transparency and user involvement; and sharing the timelock contract, multi-signer addresses, and DAO information with the public via a medium/blog link.

Finally, permanent solutions should be implemented to ensure the ongoing security and protection of the system.

- Renounce the ownership and cannot turn back to claim a privileges to execution a function

OR

- Remove a contain centralization risk function and deployed a new contract



Alleviation

BIGRIG Team has already resolved this issue by renounced ownership to `0x0000000000000000000000000000000000000000000000000000000000000000` at transaction <https://bscscan.com/tx/0x250b3e15b1a5127f75f095a554ffe364c920b01e72d34ace40faae6f27499180>

References

--



COL-01: Costly operations in a loop (costly-loop)

Vulnerabilities	Severity	Category	Status
Costly operations in a loop (costly-loop)	Informational	Best Practices	<button>Acknowledge</button>

Finding Code

✗ `LiquidityGeneratorToken.includeInReward(address)` (`src/LiquidityGeneratorToken.sol:1159-1174`) has costly operations inside a loop:

- `_excluded.pop()` (`src/LiquidityGeneratorToken.sol#1170-1173`)

Description

The contract performs **gas-expensive operations inside a loop**, such as storage reads/writes (SLOAD / SSTORE), external calls, dynamic array length reads, or complex computations. Executing these operations repeatedly within a loop can lead to **significant gas overhead**, increase transaction costs, and in worst cases cause the transaction to **run out of gas**, making the function unusable for larger input sizes.

This issue is especially problematic when loops iterate over **user-controlled or unbounded arrays**.

Impact:

- Excessive gas consumption
- Increased likelihood of out-of-gas (OOG) reverts
- Reduced scalability and usability of the contract

Recommendation

- Cache array lengths and frequently accessed values outside the loop
- Minimize SSTORE and SLOAD operations inside loops
- Avoid external calls inside loops when possible
- Consider batching or pagination for large datasets

References

Solidity Costly Operation inside a loop <https://github.com/crytic/slither/wiki/Detector-Documentation#costly-operations-inside-a-loop>



SNC-01: Conformity to Solidity naming conventions

Vulnerabilities	Severity	Category	Status
Conformity to Solidity naming conventions	Informational	Coding Style	<button>Acknowledge</button>

Finding Code

```
✗ Parameter LiquidityGeneratorToken.calculateCharityFee(uint256)._amount (src/  
LiquidityGeneratorToken.sol:1376) is not in mixedCase  
✗ Parameter LiquidityGeneratorToken.calculateLiquidityFee(uint256)._amount (src/  
LiquidityGeneratorToken.sol:1373) is not in mixedCase  
✗ Parameter LiquidityGeneratorToken.calculateTaxFee(uint256)._amount (src/LiquidityGeneratorToken.sol:  
1369-1370) is not in mixedCase  
✗ Parameter LiquidityGeneratorToken.setSwapBackSettings(uint256)._amount (src/  
LiquidityGeneratorToken.sol:1224-1225) is not in mixedCase  
✗ Variable LiquidityGeneratorToken._charityAddress (src/LiquidityGeneratorToken.sol:921) is not in  
mixedCase  
✗ Variable LiquidityGeneratorToken._charityFee (src/LiquidityGeneratorToken.sol:915-917) is not in  
mixedCase  
✗ Variable LiquidityGeneratorToken._liquidityFee (src/LiquidityGeneratorToken.sol:914) is not in  
mixedCase  
✗ Variable LiquidityGeneratorToken._taxFee (src/LiquidityGeneratorToken.sol:913) is not in mixedCase
```

Description

The smart contract does not fully adhere to Solidity's standard naming conventions. Consistent and conventional naming improves readability, maintainability, and helps avoid confusion or misinterpretation of a contract's intent.

Recommendation

It is recommended to refactor the contract to fully comply with Solidity's official naming conventions. This practice aligns with industry standards and facilitates more effective collaboration, auditing, and code comprehension.

References

Solidity Style Guide – Solidity Docs <https://docs.soliditylang.org/en/latest/style-guide.html>



UCE-01: Use Custom Errors

Vulnerabilities	Severity	Category	Status
Use Custom Errors	Informational	Gas Optimization	Acknowledge

Finding Code

```
File: LiquidityGeneratorToken.sol

154: require(owner() == _msgSender(), "Ownable: caller is not the owner");

174: require(newOwner != address(0), "Ownable: new owner is the zero address");

470: require(address(this).balance >= amount, "Address: insufficient balance");

473: require(success, "Address: unable to send value, recipient may have reverted");

543: require(address(this).balance >= value, "Address: insufficient balance for call");

544: require(isContract(target), "Address: call to non-contract");

571: require(isContract(target), "Address: static call to non-contract");

598: require(isContract(target), "Address: delegate call to non-contract");

1163: require(tAmount <= _tTotal, "Amount must be less than supply");

1188: require(!_isExcluded[account], "Account is already excluded");

1197: require(_isExcluded[account], "Account is already excluded");

1449: require(owner != address(0), "ERC20: approve from the zero address");

1450: require(spender != address(0), "ERC20: approve to the zero address");

1461: require(from != address(0), "ERC20: transfer from the zero address");

1462: require(to != address(0), "ERC20: transfer to the zero address");

1463: require(amount > 0, "Transfer amount must be greater than zero");
```

Description

The contract uses `require()` statements with revert strings to handle errors and enforce conditions. While this is syntactically valid, it is not the most efficient method in terms of gas consumption.

Starting with **Solidity 0.8.4**, developers can define and use custom errors as a more gas-efficient and structured alternative to revert strings. Custom errors reduce deployment and runtime costs by avoiding the storage of long string literals in contract bytecode.

Recommendation

Refactor `require` statements with revert strings into custom errors, particularly in frequently used functions or large codebases. This approach improves gas efficiency and allows for more structured error handling, enhancing the overall quality and professionalism of the code.



References

Control Structures – Solidity Docs <https://docs.soliditylang.org/en/latest/control-structures.html#custom-errors>



LRS-01: Use of Long Revert Strings in `require()` or `revert()`

Vulnerabilities	Severity	Category	Status
Use of Long Revert Strings in `require()` or `revert()`	Informational	Gas Optimization	Acknowledge

Finding Code

```
File: LiquidityGeneratorToken.sol

174: require(newOwner != address(0), "Ownable: new owner is the zero address");

473: require(success, "Address: unable to send value, recipient may have reverted");

543: require(address(this).balance >= value, "Address: insufficient balance for call");

571: require(isContract(target), "Address: static call to non-contract");

598: require(isContract(target), "Address: delegate call to non-contract");

1449: require(owner != address(0), "ERC20: approve from the zero address");

1450: require(spender != address(0), "ERC20: approve to the zero address");

1461: require(from != address(0), "ERC20: transfer from the zero address");

1462: require(to != address(0), "ERC20: transfer to the zero address");

1463: require(amount > 0, "Transfer amount must be greater than zero");
```

Description

The contract uses long revert strings in `require()` or `revert()` statements. While these strings can improve debugging during development, they increase the bytecode size and gas costs at both deployment and runtime.

In Ethereum, revert strings are stored in the bytecode and transmitted with the transaction when a revert occurs. Longer strings consume more storage and gas, which may lead to higher transaction fees and even deployment size limitations in extreme cases.

Recommendation

Long revert strings in the `require` statements consume more gas. It's more gas-efficient to use short revert strings or error codes. In addition, using custom errors can further optimize gas usage while providing clear and meaningful error messages.

Replace long revert strings with:

- Shorter, concise messages when strings are necessary.
- Preferably, use custom errors to achieve optimal gas efficiency while maintaining readability.

References

Solidity Docs – Custom Errors: <https://docs.soliditylang.org/en/latest/control-structures.html#custom-errors>



USR-01: Use shift Right/Left instead of division/multiplication if possible

Vulnerabilities	Severity	Category	Status
Use shift Right/Left instead of division/multiplication if possible	Informational	Gas Optimization	Acknowledge

Finding Code

```
File: LiquidityGeneratorToken.sol  
923: uint256 public constant MAX_FEE = 10**4 / 4;
```

Description

Solidity supports bitwise shift operations (<< and >>) which are **more gas-efficient** than using arithmetic multiplication or division when the operation involves powers of two. Using $x * 2$ or $x / 2$ can be replaced with $x << 1$ or $x >> 1$, respectively, for better performance and lower gas costs.

Recommendation

When multiplying or dividing by powers of two, replace the arithmetic operation with bitwise shift operators.

Example, This is not a fixed code

✓ Preferred:

```
uint256 result = value << 1; // equivalent to value * 2  
uint256 result = value >> 2; // equivalent to value / 4
```

✗ Avoid:

```
uint256 result = value * 2;  
uint256 result = value / 4;
```



UIC-01: Use != 0 instead of > 0 for unsigned integer comparison

Vulnerabilities	Severity	Category	Status
Use != 0 instead of > 0 for unsigned integer comparison	Informational	Gas Optimization	Acknowledge

Finding Code

```
File: LiquidityGeneratorToken.sol
382: require(b > 0, errorMessage);
408: require(b > 0, errorMessage);
450: return size > 0;
619: if (returndata.length > 0) {
1189: if (_rOwned[account] > 0) {
1391: if (tCharity > 0) {
1463: require(amount > 0, "Transfer amount must be greater than zero");
```

Description

The contract uses `> 0` to check whether an unsigned integer (uint) is non-zero. While functionally correct, this comparison is slightly less efficient than using `!= 0`, due to the way EVM opcodes are generated and optimized.

In the EVM, the `!=` operation is generally compiled to fewer instructions than `>`, resulting in minor gas savings. Since uint variables cannot be negative, both expressions yield the same result, but `!= 0` is considered a more gas-optimal and idiomatic choice for checking non-zero conditions.

Recommendation

Use `!= 0` instead of `> 0` when checking whether a `uint` is non-zero. This is a gas-efficient and stylistically preferred approach in Solidity.



A Appendix

A.1 Source Units in Scope

Source Units Analyzed: 1
Source Units in Scope: 1 (100%)

Type	File	Logic Contracts	Interfaces	Lines	nLines	nSLOC	Comment Lines	Complex. Score	Capabilities
	src/ LiquidityGeneratorToken.sol	6	4	1643	1187	724	396	499	 *****Σ
	Totals	6	4	1643	1187	724	396	499	 *****Σ



A.2 Visibility, Mutability, Modifier function testing

Components

 ****Contracts	 ****Libraries	 ****Interfaces	 ****Abstract
1	2	4	3

Exposed Functions

This section lists functions that are explicitly declared public or payable. Please note that getter methods for public stateVars are not included.

 ****Public	 ****Payable
66	6

External	Internal	Private	Pure	View
44	84	24	20	34

StateVariables

Total	 ****Public
28	9

Capabilities

Solidity Versions observed	 ** Experimental Features**	 ** Can Receive Funds**	 ** Uses Assembly**	 ** Has Destroyable Contracts**
=0.8.4		yes	yes (2 asm blocks)	***

 ** Transfers ETH**	 **** Low-Level Calls	 ** DelegateCall**	 ** Uses Hash Functions**	 ** ECRecover**	 ** New/Create/Create2**
yes	***	yes	***	***	***

 ** TryCatch**	Σ Unchecked
***	yes

Dependencies / External Imports

Dependency / Import Path	Count
--------------------------	-------



A.3 Contracts Description Table

Contracts Description Table

Contract	Type	Bases		
↳	Function Name	Visibility	Mutability	Modifiers
IERC20	Interface			
↳	totalSupply	External !		NO !
↳	balanceOf	External !		NO !
↳	transfer	External !	●	NO !
↳	allowance	External !		NO !
↳	approve	External !	●	NO !
↳	transferFrom	External !	●	NO !
Context	Implementation			
↳	_msgSender	Internal 🔒		
↳	_msgData	Internal 🔒		
Ownable	Implementation	Context		
↳		Public !	●	NO !
↳	owner	Public !		NO !
↳	renounceOwnership	Public !	●	onlyOwner
↳	transferOwnership	Public !	●	onlyOwner
↳	_setOwner	Private 🔒	●	
SafeMath	Library			
↳	tryAdd	Internal 🔒		
↳	trySub	Internal 🔒		
↳	tryMul	Internal 🔒		
↳	tryDiv	Internal 🔒		



Contract	Type	Bases		
L	tryMod	Internal 🔒		
L	add	Internal 🔒		
L	sub	Internal 🔒		
L	mul	Internal 🔒		
L	div	Internal 🔒		
L	mod	Internal 🔒		
L	sub	Internal 🔒		
L	div	Internal 🔒		
L	mod	Internal 🔒		
Address	Library			
L	isContract	Internal 🔒		
L	sendValue	Internal 🔒	🔴	
L	functionCall	Internal 🔒	🔴	
L	functionCall	Internal 🔒	🔴	
L	functionCallWithValue	Internal 🔒	🔴	
L	functionCallWithValue	Internal 🔒	🔴	
L	functionStaticCall	Internal 🔒		
L	functionStaticCall	Internal 🔒		
L	functionDelegateCall	Internal 🔒	🔴	
L	functionDelegateCall	Internal 🔒	🔴	
L	verifyCallResult	Internal 🔒		
IUniswapV2Router01	Interface			
L	factory	External !		NO !
L	WETH	External !		NO !
L	addLiquidity	External !	🔴	NO !
L	addLiquidityETH	External !	🟢	NO !
L	removeLiquidity	External !	🔴	NO !



Contract	Type	Bases		
L	removeLiquidityETH	External !	●	NO !
L	removeLiquidityWithPermit	External !	●	NO !
L	removeLiquidityETHWithPermit	External !	●	NO !
L	swapExactTokensForTokens	External !	●	NO !
L	swapTokensForExactTokens	External !	●	NO !
L	swapExactETHForTokens	External !	8	NO !
L	swapTokensForExactETH	External !	●	NO !
L	swapExactTokensForETH	External !	●	NO !
L	swapETHForExactTokens	External !	8	NO !
L	quote	External !		NO !
L	getAmountOut	External !		NO !
L	getAmountIn	External !		NO !
L	getAmountsOut	External !		NO !
L	getAmountsIn	External !		NO !
IUniswapV2Router02	Interface	IUniswapV2Router01		
L	removeLiquidityETHSupportingFeeOnTransferTokens	External !	●	NO !
L	removeLiquidityETHWithPermitSupportingFeeOnTransferTokens	External !	●	NO !
L	swapExactTokensForTokensSupportingFeeOnTransferTokens	External !	●	NO !
L	swapExactETHForTokensSupportingFeeOnTransferTokens	External !	8	NO !
L	swapExactTokensForETHSupportingFeeOnTransferTokens	External !	●	NO !
IUniswapV2Factory	Interface			
L	feeTo	External !		NO !
L	feeToSetter	External !		NO !
L	getPair	External !		NO !
L	allPairs	External !		NO !



Contract	Type	Bases		
L	allPairsLength	External !		NO !
L	createPair	External !	●	NO !
L	setFeeTo	External !	●	NO !
L	setFeeToSetter	External !	●	NO !
BaseToken	Implementation			
LiquidityGeneratorToken	Implementation	IERC20, Ownable, BaseToken		
L		Public !	✅	NO !
L	name	Public !		NO !
L	symbol	Public !		NO !
L	decimals	Public !		NO !
L	totalSupply	Public !		NO !
L	balanceOf	Public !		NO !
L	transfer	Public !	●	NO !
L	allowance	Public !		NO !
L	approve	Public !	●	NO !
L	transferFrom	Public !	●	NO !
L	increaseAllowance	Public !	●	NO !
L	decreaseAllowance	Public !	●	NO !
L	isExcludedFromReward	Public !		NO !
L	totalFees	Public !		NO !
L	deliver	Public !	●	NO !
L	reflectionFromToken	Public !		NO !
L	tokenFromReflection	Public !		NO !
L	excludeFromReward	Public !	●	onlyOwner
L	includeInReward	External !	●	onlyOwner



Contract	Type	Bases		
L	_transferBothExcluded	Private 🔒	●	
L	excludeFromFee	Public !	●	onlyOwner
L	setTaxFeePercent	External !	●	onlyOwner
L	setLiquidityFeePercent	External !	●	onlyOwner
L	setCharityFeePercent	External !	●	onlyOwner
L	setSwapBackSettings	External !	●	onlyOwner
L		External !	🟢	NO !
L	_reflectFee	Private 🔒	●	
L	_getValues	Private 🔒		
L	_getTValues	Private 🔒		
L	_getRValues	Private 🔒		
L	_getRate	Private 🔒		
L	_getCurrentSupply	Private 🔒		
L	_takeLiquidity	Private 🔒	●	
L	_takeCharityFee	Private 🔒	●	
L	calculateTaxFee	Private 🔒		
L	calculateLiquidityFee	Private 🔒		
L	calculateCharityFee	Private 🔒		
L	removeAllFee	Private 🔒	●	
L	restoreAllFee	Private 🔒	●	
L	isExcludedFromFee	Public !		NO !
L	_approve	Private 🔒	●	
L	_transfer	Private 🔒	●	
L	swapAndLiquify	Private 🔒	●	lockTheSwap
L	swapTokensForEth	Private 🔒	●	



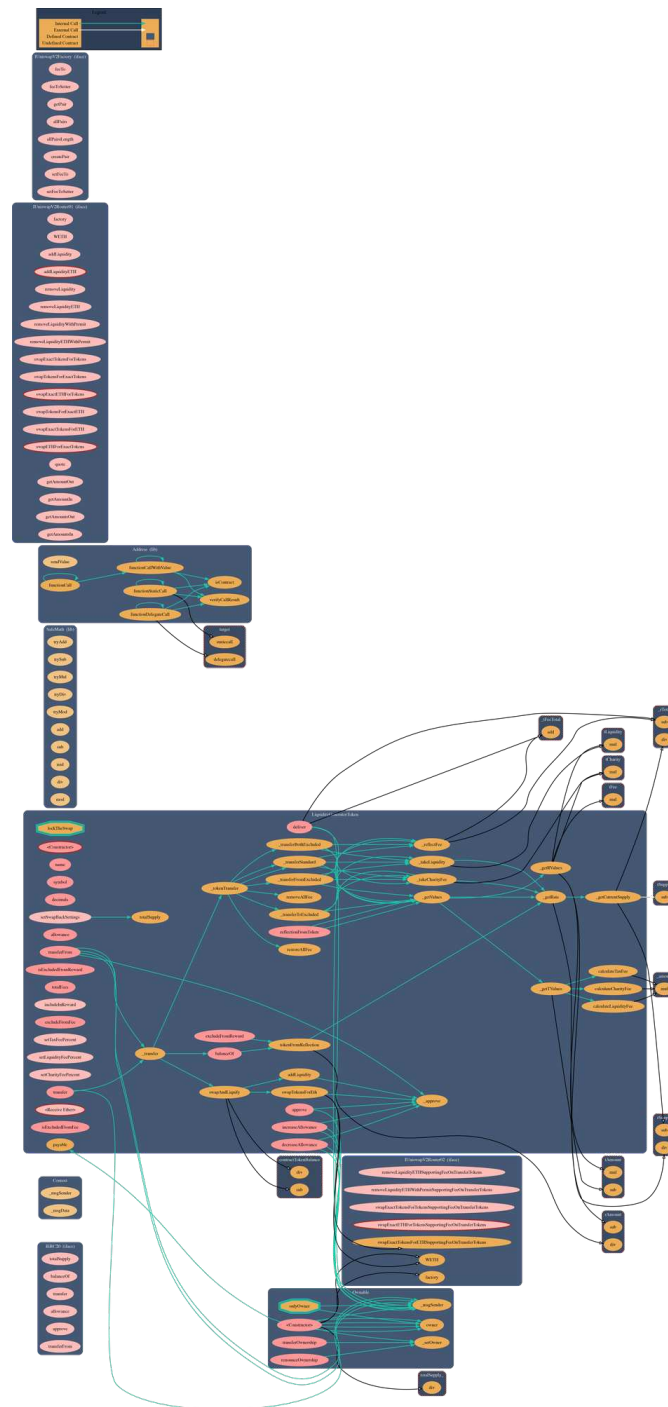
Contract	Type	Bases		
L	addLiquidity	Private 		
L	_tokenTransfer	Private 		
L	_transferStandard	Private 		
L	_transferToExcluded	Private 		
L	_transferFromExcluded	Private 		

Legend

Symbol	Meaning
	Function can modify state
	Function is payable



A.4 Call Graph



A.5 UML Class Diagram



About SCRL

For partner

Proven Cybersecurity Expertise



Highlight



Cyber-attacks are ineffective when our customers utilize our services.

Our customers enjoy the highest level of cybersecurity, and we have successfully prevented any significant cyber attacks against them. Over the past three years, we have dedicated significant efforts to safeguarding our customers' data and systems.



Cybersecurity Assessment and Penetration Testing by Multiple Experts

Our team comprises experienced security researchers and penetration testers with expertise in Web3 and Blockchain, as well as general industries. Our primary focus is conducting comprehensive security assessments to provide secure solutions to our valued customers.



Collaborative law enforcement investigations

We have provided assistance to victims in numerous cases, including the Pig-butcher Scam, Hybrid Scam. Through collaborative efforts with law enforcement agencies, we have successfully secured justice through the court system.

SCRL is a dynamic cybersecurity team that provides security assessments, penetration testing, and incident investigations. We utilize specialized tools, expertise, and frameworks to enhance security measures. Additionally, we develop robust internal tools tailored to our organization's needs.

SCRL is committed to driving security for the blockchain industry and businesses alike.

Follow Us On:

Platform	Link
Website	https://scrl.io/
Twitter	https://twitter.com/scrl_io
Telegram	https://t.me/scrl_io
Medium	https://scrl.medium.com/

